

1.1 definitie

wat is het verschil tussen microprocessors en een microcontroller?

Microprocessor

Ook wel gekend als een CPU (central processing unit) geeft bijna nooit een interne geheugen module met uitzondering van cachegeheugen en registers.

Het heeft daarom altijd externe componenten nodig voor opslag en I/O.

Bevat een ALU (Arithmetic logic unit) deze zorgt voor de wiskundige bewerkingen en logische poort bewerkingen (and, or, not).

De data dient van extern geheugen te komen en de resultaten worden naar extern geheugen geschreven.

Denk hierbij aan een computer

Het geheugen bestaat meestal uit 2 delen :

- 1 om de variabelen van en naar toe te schrijven
- 2 om de code voor het programma zelf in op te slaan

door verschil in snelheid tussen cpu en geheugen is er optimalisatie nodig

1^{ste}

Registers

Data wordt in stappen gekopieerd naar geheugen eerst naar registers in registerbanken bevinden zich meestal dicht bij de cpu.

Deze zijn tijdelijke geheugens en zijn veel sneller dan geheugenmemory maar zijn wel beperkt in grootte en aantal.

De ALU werkt met de data die in de registers zit om snel te werken en daarna terug in de registers te schrijven daarna word het uit de registers terug in geheugen geschreven.

2^{de}

Cache geheugen

Dit is een tussenslag tussen registers en memory

Data die veel opgevraagd word zal hierin opgeslagen worden.

Deze is trager dan registers maar sneller dan memory en is groter dan registers maar kleiner dan memory.

Wordt meestal in levels opgedeeld typisch

L1 kleinste en snelste

L2 midden slag

L3 grootste en trager

Er zijn al cpu die lvl4 cache geheugen hebben

Microcontroller

Ook wel MCU genoemd (microcontroller unit)

Is een compacte van alles voorzien systeem met bijkomende perefierie aka modules.

Meestal verbonden door bussen .

Enkele perefierie:

- Digitale IO
- Analoge IO
- Timers
- Seriële interfaces (UART, SPI, I²C,...)
- Usb
- Can bus
- Ethernet

Embedded system

Een **embedded system** is een mini-computer die speciaal ontworpen is om één specifieke taak te doen, ingebouwd in een groter apparaat.

Sammenvating microcontrollers

Kenmerk	CPU	Microcontroller
Wat is het?	Een krachtige rekenkern die deel uitmaakt van een computersysteem	Een alles-in-één chip met CPU, geheugen en I/O voor specifieke taken
Interne componenten	Alleen de processor zelf	Processor + RAM + ROM/Flash + I/O + timers + ADC enz.
Externe componenten	Vereist aparte RAM, opslag en randapparatuur	Alles is geïntegreerd op één chip
Kloksnelheid	Hoog (GHz, vaak multi-core)	Lager (kHz – MHz, meestal single-core)
Vermogenverbruik	Hoog (meerdere watts)	Laag (milliwatts tot <1 watt)
Gebruik	Algemene computers, laptops, servers	Ingebedde systemen zoals afstandsbedieningen, sensoren, robotica
Taken	Meerdere complexe processen tegelijk	Specifieke, eenvoudige of real-time taken
Voorbeelden	Intel Core i5/i7, AMD Ryzen, Apple M1	STM32, ATmega328 (Arduino), ESP32, PIC
Besturingssysteem	Draait meestal een volledig OS (Windows, Linux, macOS)	Vaak geen OS, of een eenvoudig RTOS of bare-metal code
Programmeertaal	Hoog niveau: Python, Java, C#, enz.	Lager niveau: C, C++, Assembly
Krachtig	☑	✗ (vooral voor simpele taken)
Energiezuinig	✗	☑
Alles-in-één	✗	☑
Kosten	Duurder	Goedkoper
Toepassingen	Algemeen gebruik	Specifieke embedded toepassingen

SoC (system on chip)

Een **SoC** is als een hele computer op één chip, met alles erop en eraan — ideaal voor compacte, krachtige apparaten zoals smartphones, Raspberry Pi's en slimme IoT-apparaten.

Onderdelen

CPU = de brein van het systeem

Oscillator = maakt met een kristal een klok 'hartslag' voor het systeem

Bussen = zijn de 'zenuwbanen' van het systeem waar data over loopt

ROM (readonly memory) / Flash :

De locatie waar het programma wordt opgeslagen en is niet volatiel

(data is niet verloren als systeem zonder spanning komt)

RAM (random access memory) :

Geheugen die volatiel is en zeer snel

(verlies zijn dat als het systeem zonder spanning komt)

Timers = hardware die tellen en signalen geven na een bepaalde tijd

Interupt controllers = controleerd interrupts die dan naar de cpu worden gestuurd

Sammenvating microcontrollers

H5

GPIO

Bij reset worden alle pinnen op input gezet dit omdat ze dan hoog impedant zijn.

Dit omdat ze dan geen grote belasting voor de aangesloten hardware vormen.

Pinnen

Worden aangeduid met de letter P en dan de bus waartoe ze behoren (A...F).

Er zijn max 16 pinnen per bus dus men kan max tot 80 pinnen gaan.

Er zijn ook andere pin namen die bv beginnen met de letter A/D deze zijn bv arduino pin-headers en kunnen niet door de CMSIS-lib gebruikt worden.

Operators

Operator	Naam	Uitleg	Voorbeeld gebruik
&	AND	Alleen 1 als beide bits 1 zijn	Maskeren: value & 0x0F
	OR	1 als minstens één bit 1 is	Bit aanzetten: REG
^	XOR	1 als bits verschillend zijn	Bit toggelen: REG ^= (1 << x)
~	NOT	Keert alle bits om (1 → 0, 0 → 1)	Inversie: ~value
<<	Shift Left	Verplaatst bits naar links, vult met nullen rechts	(1 << 3) = 0b00001000
>>	Shift Right	Verplaatst bits naar rechts, vult met nullen links (of behoudt teken)	value >> 2

Wanneer gebruik je welke operator?

1. & (AND)

Gebruik voor:

- **Bits testen / uitlezen**
- **Maskers toepassen** (alleen bepaalde bits behouden)

Voorbeeld:

```
if (REG & (1 << 5)) {  
    // Bit 5 is actief (1)  
}
```

Waarom: je wil **controleren of een specifieke bit aanstaat**.

2. | (OR)

Gebruik voor:

- **Bits aanzetten** zonder andere te wijzigen

Voorbeeld:

```
REG |= (1 << 2); // Zet bit 2 aan
```

Waarom: je wil **een of meerdere bits instellen op 1** zonder andere aan te raken.

3. ^ (XOR)

Gebruik voor:

- **Bits omdraaien (toggelen)**

Voorbeeld:

```
REG ^= (1 << 3); // Toggle bit 3
```

Waarom: je wil een bit wisselen tussen 1 en 0 telkens als het wordt aangeroepen.

4. ~ (NOT)

Gebruik voor:

- Alle bits inverteren

Voorbeeld:

```
uint8_t x = 0b00001111;  
uint8_t y = ~x; // y = 0b11110000
```

Waarom: handig voor bijvoorbeeld **complementwaarden** of **alle bits wissen behalve een masker**.

5. << (Shift Left)

Gebruik voor:

- Bit op bepaalde positie zetten
- Vermenigvuldigen met machten van 2

Voorbeeld:

```
REG |= (1 << 6); // Zet bit 6 aan
```

Waarom: zet een 1 op exact de juiste bitpositie.

6. >> (Shift Right)

Gebruik voor:

- Delen door machten van 2
- Bits opschuiven naar rechts

Voorbeeld:

```
value = value >> 1; // Deel door 2
```

Waarom: bijv. voor **snelle deling** of **uitlezen van bepaalde bits**.

Sammenvating microcontrollers

a = 5 en b = 3

Operator	Naam	Beschrijving	Voorbeeld
&	Bitwise AND	Zet bits op 1 als beide operands 1 hebben op die plek.	<code>int a = 5, b = 3; int c = a & b; // c = 1 (0101 & 0011 = 0001)</code>
	Bitwise OR	Zet bits op 1 als minstens één operand 1 heeft.	<code>int c = a b; // c = 7 (0101 0011 = 0111)</code>
^	Bitwise XOR	Zet bits op 1 als precies één operand 1 heeft.	<code>int c = a ^ b; // c = 6 (0101 ^ 0011 = 0110)</code>
~	Bitwise NOT	Keert alle bits om (1 wordt 0, 0 wordt 1).	<code>int c = ~a; // c = -6 (0101 wordt 1010 in 2's complement)</code>
<<	Links shift	Schuift bits naar links, vult met nullen aan de rechterkant.	<code>int c = a << 1; // c = 10 (0101 << 1 = 1010)</code>
>>	Rechts shift	Schuift bits naar rechts, vult afhankelijk van teken.	<code>int c = a >> 1; // c = 2 (0101 >> 1 = 0010)</code>
&=	Bitwise AND + toewijzing	Doet AND en slaat resultaat op in de linkeroperand.	<code>a &= b; // a wordt 1 (0101 & 0011 = 0001)</code>
=	Bitwise OR + toewijzing	Doet OR en slaat resultaat op in de linkeroperand.	<code>a = b; // a wordt 7 (0101 0011 = 0111)</code>
^=	Bitwise XOR + toewijzing	Doet XOR en slaat resultaat op in de linkeroperand.	<code>a ^= b; // a wordt 6 (0101 ^ 0011 = 0110)</code>
<<=	Links shift + toewijzing	Schuift links en slaat resultaat op in de linkeroperand.	<code>a <<= 1; // a wordt 10 (0101 << 1 = 1010)</code>
>>=	Rechts shift + toewijzing	Schuift rechts en slaat resultaat op in de linkeroperand.	<code>a >>= 1; // a wordt 2 (0101 >> 1 = 0010)</code>

Men kan voor deze operaties eigenlijk kiezen wat men zet

Vergelijking met andere notaties

- **Decimaal:** `int x = 5;` (geen prefix).
- **Hexadecimaal:** `int x = 0x5;` (prefix `0x`).
- **Octaal:** `int x = 05;` (prefix `0`).
- **Binair:** `int x = 0b0101;` (prefix `0b`).

Men opteert natuurlijk het liefst om met bits te werken omdat dit handiger is dan het deciaal of hexadecimaal systeem (men kan duidelijker zien welke bit wordt aangelegd)

Sammenvating microcontrollers

Men gebruikt voor de schift operatoren het best de decimale notatie omdat men dan een **beperkt aantal** bits te kunnen manipuleren.

Bv

$| = (1 \ll 6) = \text{ob}0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0100\ 0000 = 64 = 0x40 = 0100$

$| = (1 \ll 19) = \text{ob}0000\ 0000\ 0000\ 1000\ 0000\ 0000\ 0000\ 0000 = 524\ 288 = 0x8000 = 02000000$

0b = binaire notatie

Decimaale notatie

Hex notatie

Octaal notatie

alle notatie zijn juist maar afhankelijk van wat je wilt doen zal je de gemakkelijkste notatie kiezen.

Men gebruikt de hex notatie als men **alle** bits wil manipuleren in een register daar deze meestal 32 bits bevat

Binair word minder gebruikt omdat deze door het 32 bit formaat zeer groot word en onoverzichtelijk. (tenzij het kleine getallen zijn)